

WELCOME

COOKED



K02
KELOMPOK N

RaFeyFa Dea {18224044}
Zahra Zahidah P {18224038}

Zahrah Nur A {18224092}
Athar Ramadhana {18224104}

USER MANUAL

Nimon's Cooked adalah permainan versi sederhana yang berasal dari game Overcooked. Permainan ini hanya dapat dimainkan oleh satu player, tetapi akan ada dua chef dalam satu permainan. Player dapat bermain spesifik di map burger. Player akan mendapat waktu satu menit untuk tiap order yang perlu diselesaikan. Untuk memenangkan permaiann, player perlu memperoleh 800 poin.



GAME DESC

1. Clone repository GitHub
2. Buka Folder Nimonscooked
3. Buka Folder menggunakan IDE seperti Visual Studio Code atau IntelliJIDEA
4. Klik tombol Run Java
5. Pilih Game.java
6. Selamat bermain!

- W, A, S, D : menggerakkan chef player ke atas, bawah, kanan, kiri
- V: pick/drop item yang berada di hadapan chef player
- B: switch chef player yang aktif



HOW TO PLAY

GAMEPLAY

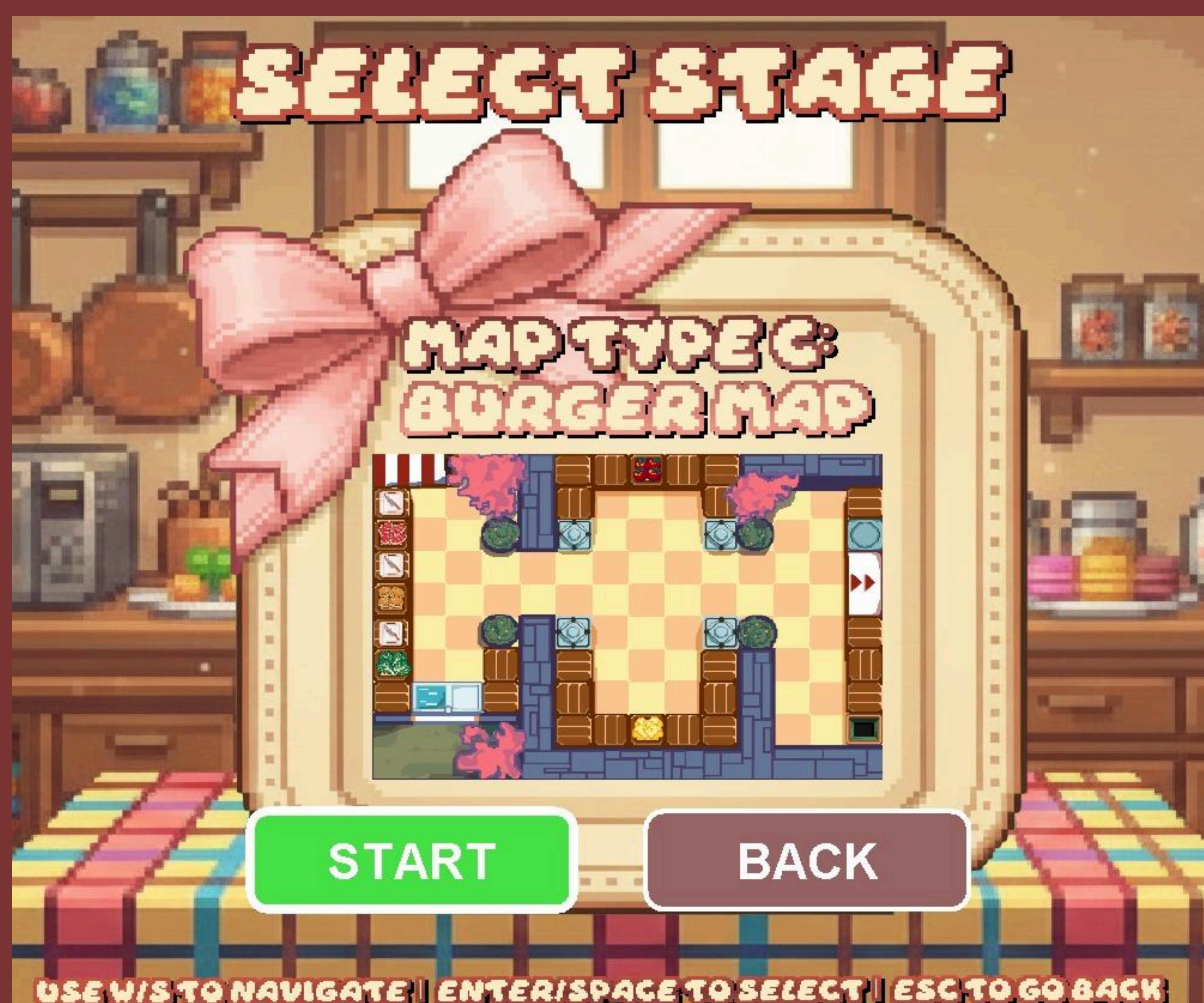
MAIN

MENU



Saat program dijalankan, tampilan pertama yang akan muncul adalah Main Menu. Pada tampilan ini, player dapat memilih **start**, **how to play**, **credits**, atau **exit**.

Player dapat menekan tombol **START** untuk memulai permainan atau tombol **BACK** untuk kembali ke main menu.



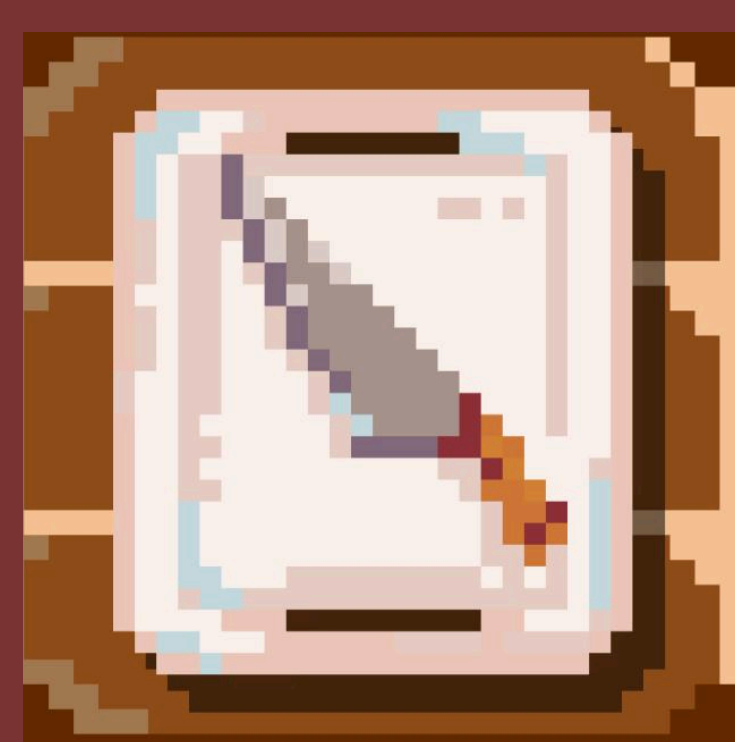
SELECT STAGE

MAP AND STATION



Tipe map yang digunakan adalah Map Type 0 atau **BURGER MAP**, Map ini menuntut ritme kerja yang cepat karena daging harus dipotong lalu digoreng sementara alat masak yang digunakan terbatas.

CUTTING STATION



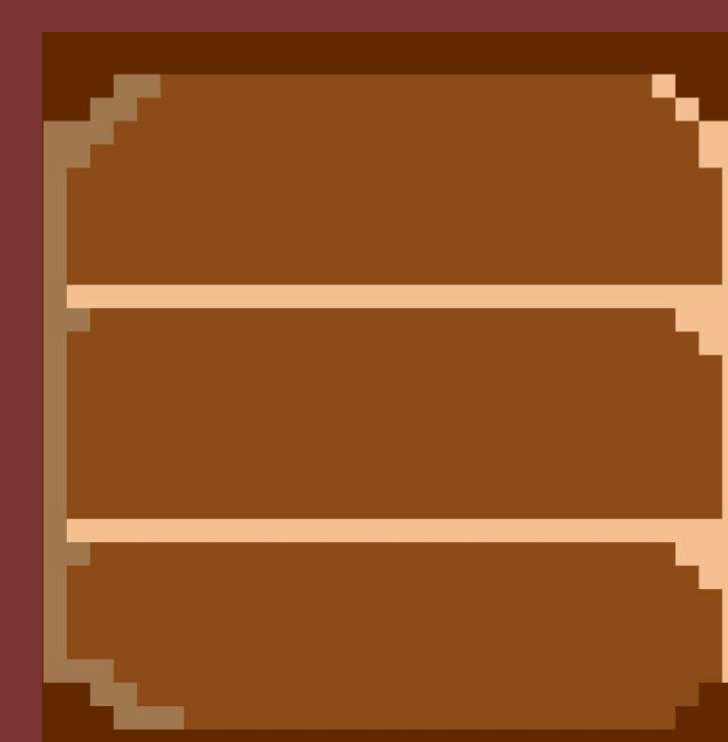
Memotong bahan mentah menjadi siap olah

INGREDIENT STORAGE



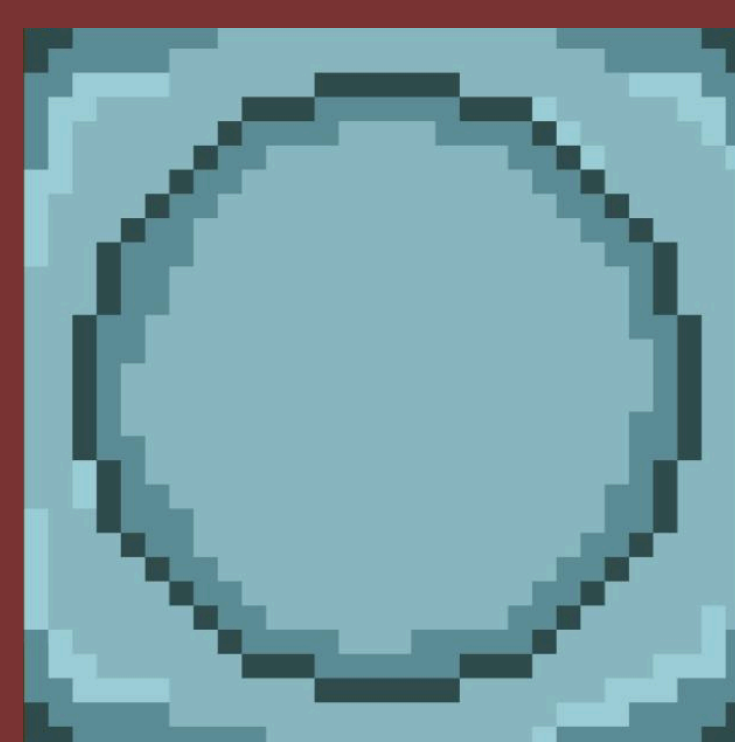
Menyediakan stok bahan mentah tak terbatas

ASSEMBLY STATION



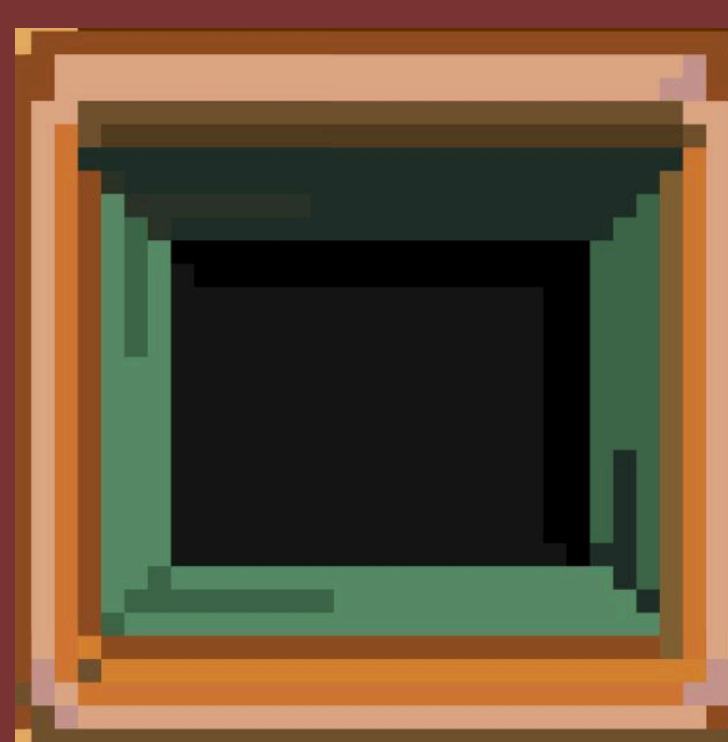
Menyediakan permukaan kosong untuk menaruh bahan dan merakit hidangan.

PLATE STATION



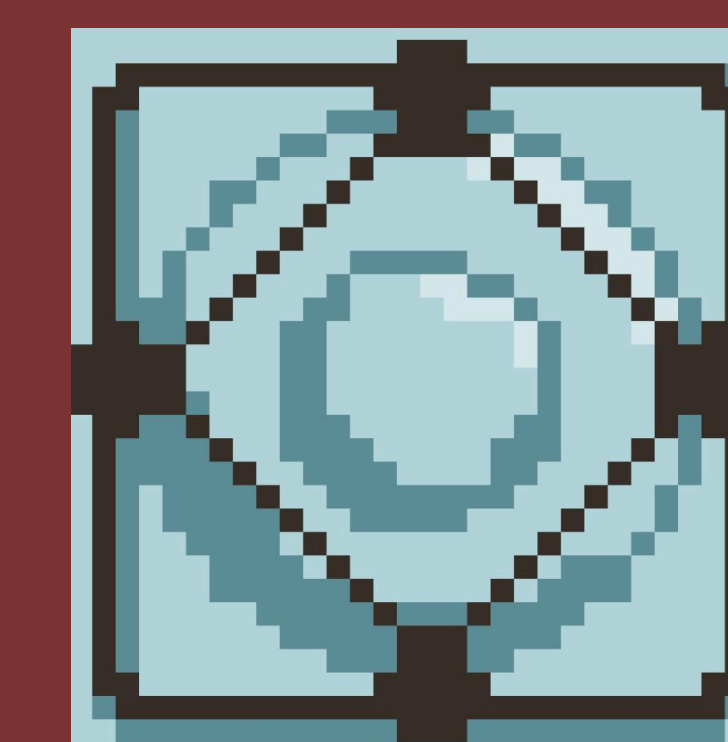
Menampung plate bersih dan kotor, tidak menerima drop item

TRASH STATION



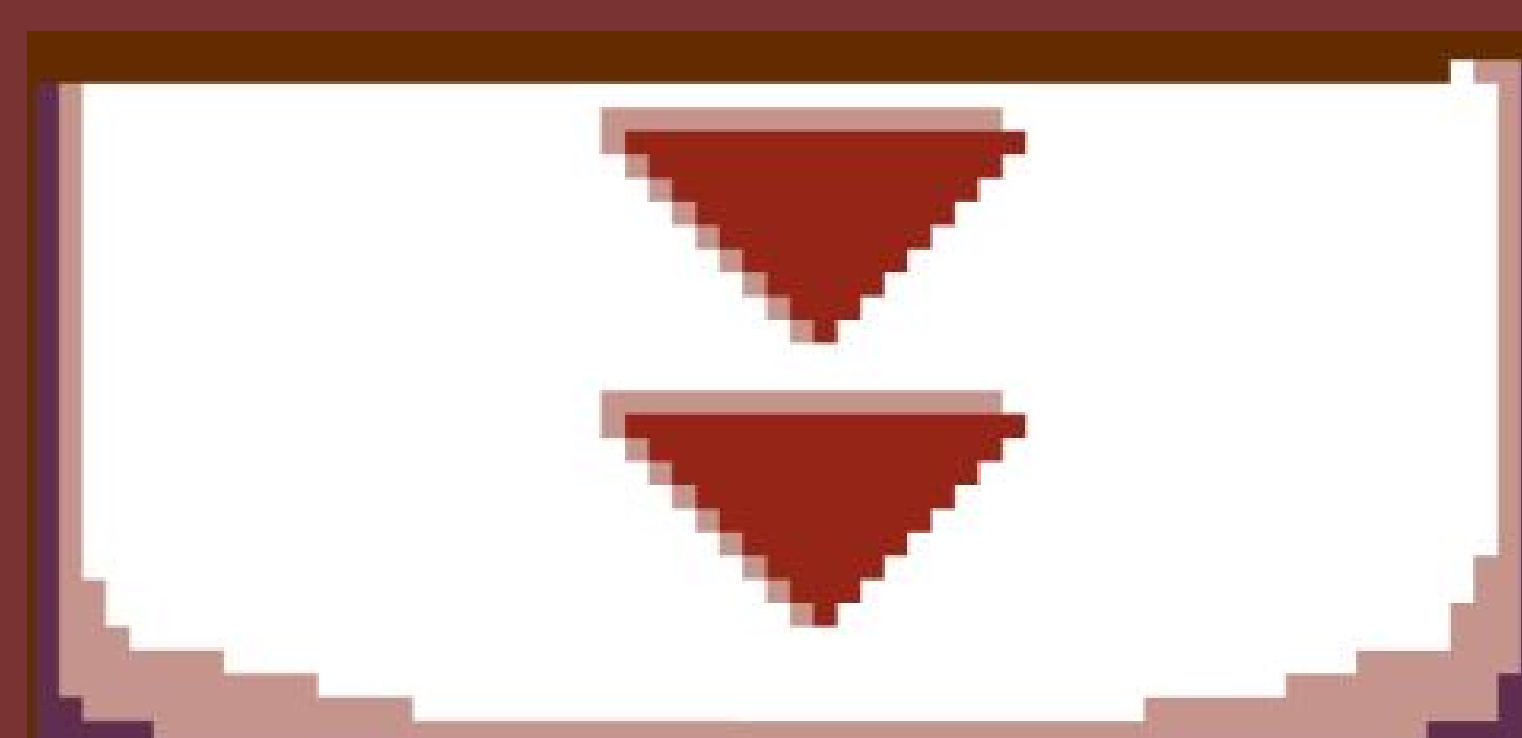
Membuang bahan atau isi hidangan yang tidak dibutuhkan

COOKING STATION



Memasak bahan secara otomatis menjadi matang

SERVING STORAGE



Menyajikan dish selesai dengan plate kepada pelanggan untuk divalidasi

WASHING STATION



Mencuci plate kotor, progres mencuci dapat dilanjutkan kembali jika ditinggalkan

RECIPE

CLASSIC BURGER



CHEESE BURGER



BLT BURGER



DELUXE BURGER



GAME FLOW

1



Ambil item di ingredient storage

2



Potong bahan mentah di cutting station

3



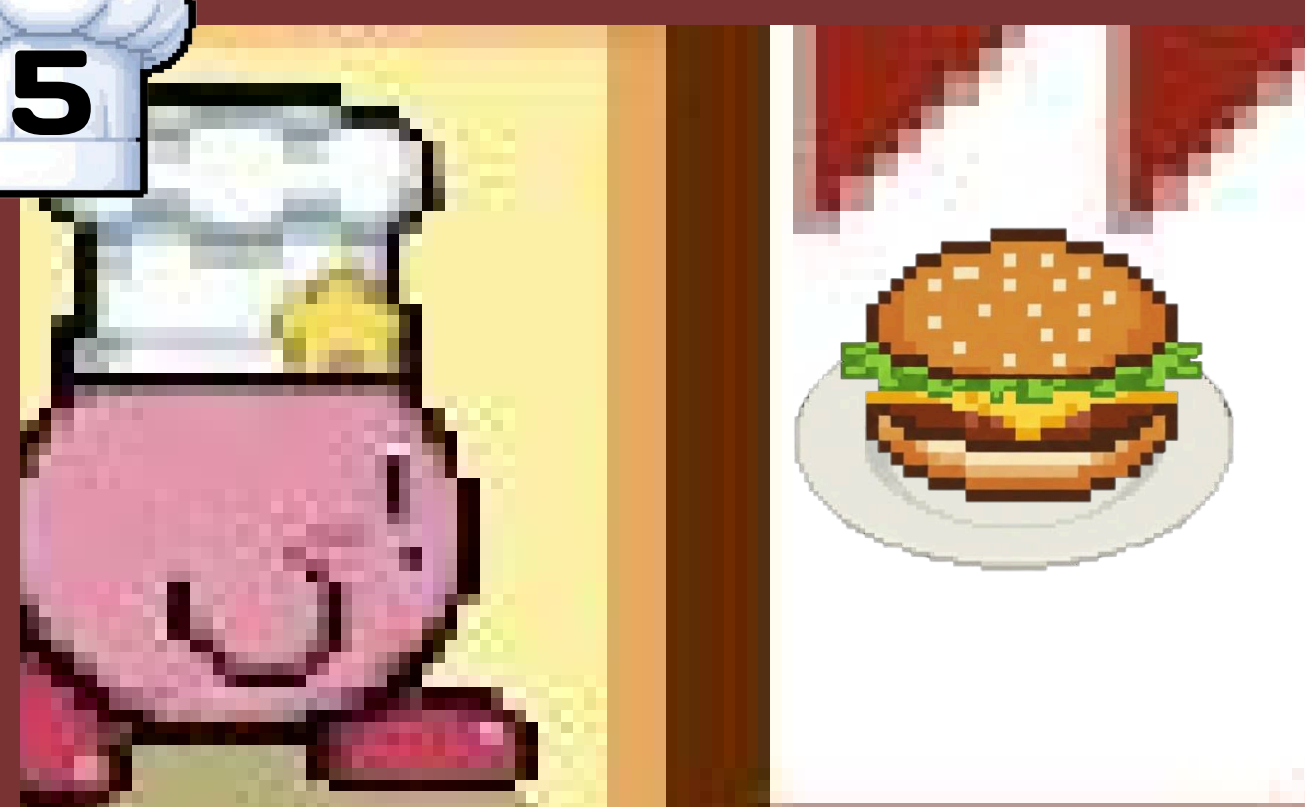
Masak bahan yang telah dipotong selama 12 detik

4



Ambil piring bersih dan susun bahan

5



Serve ke customer sesuai order

6



Cuci piring kotor setelah serving

RULES



Player mengendalikan dua chef, tetapi hanya satu chef yang aktif dan dapat digerakkan pada satu waktu.



Jika tangan kosong, ambil item di depan. Jika tangan memegang item, letakkan di tile kosong di depan



Chef hanya dapat memegang satu item dalam satu waktu. Piring kotor yang ditumpuk dapat dibawa seluruhnya.



Jika Chef yang ditinggalkan sedang busy (memotong/mencuci), ia akan menyelesaikan pekerjaannya sendiri.

WIN/LOSE CONDITION



TIME'S UP!

Timer countdown mencapai 0. Total skor dihitung, dan hasil dievaluasi (*Pass/Fail*) berdasarkan batas minimum yang telah ditentukan.



WIN CONDITION



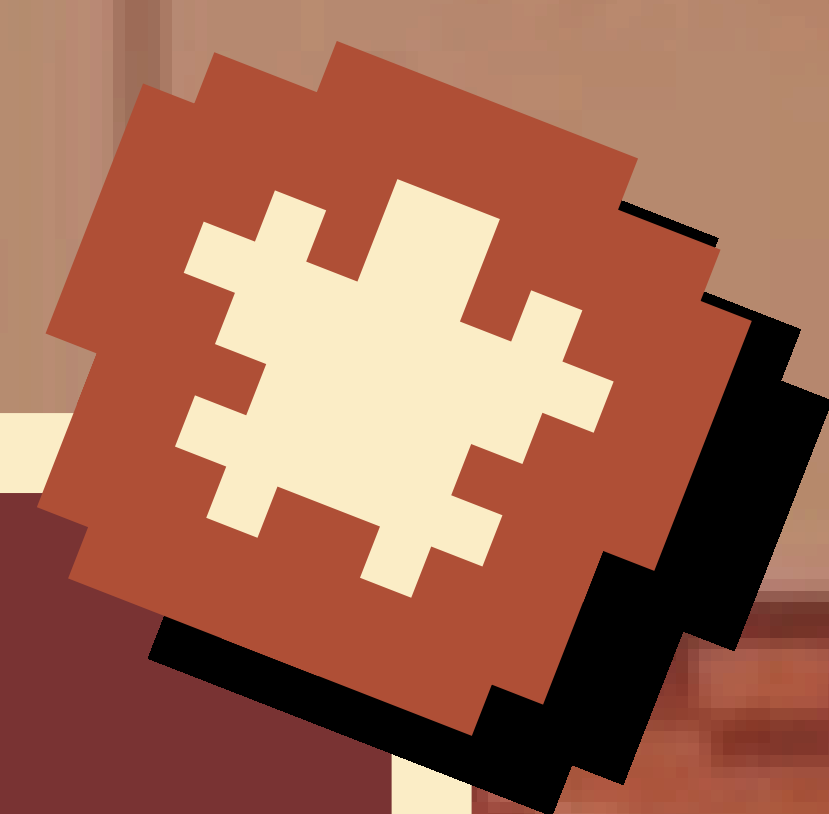
TOO MANY FAILED ORDERS

Player gagal menyelesaikan sejumlah order secara berturut-turut hingga skor menjadi negatif.

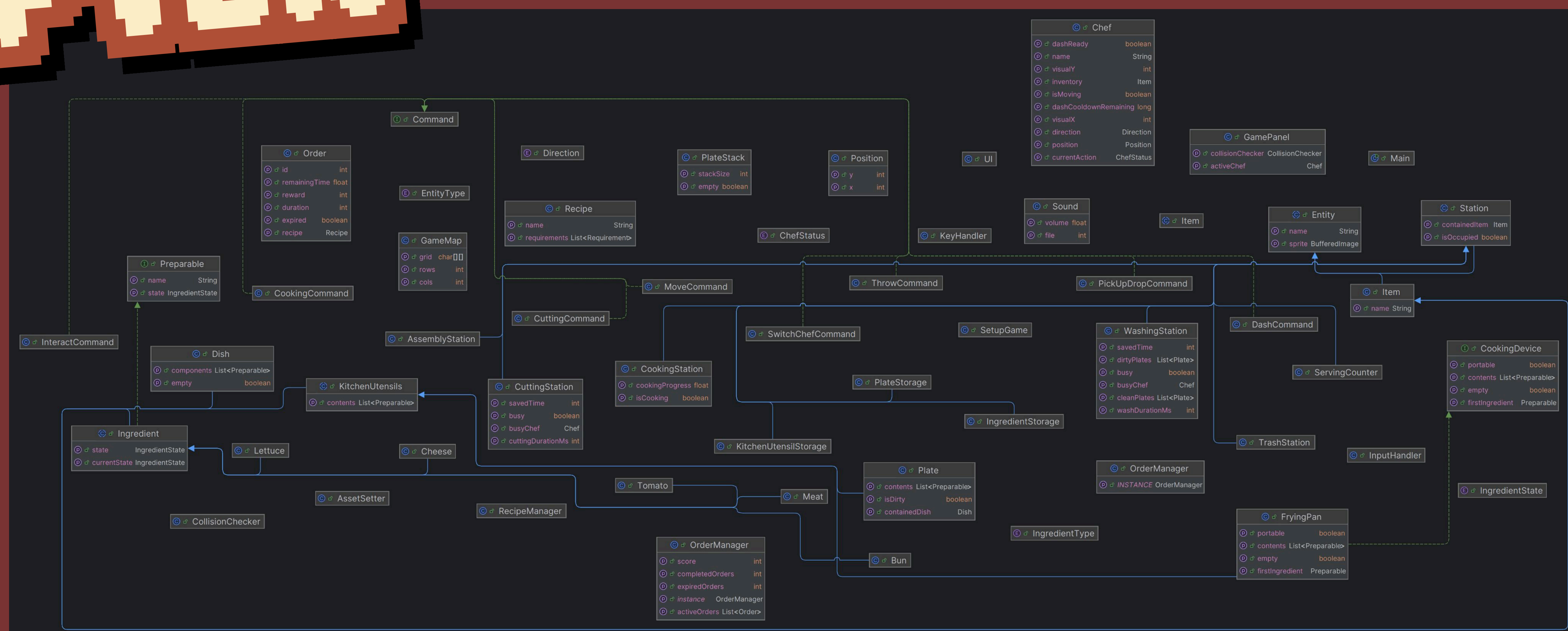


LOSE CONDITION

BEFORE



AFTER



Berikut adalah struktur Anal dari objek-objek yang terdapat pada game. (Full: <https://bit.ly/GlassDiagramNimonscooked>)

DESIGN PATTERN

APA ITU DESIGN PATTERN?

Design Pattern merupakan cara yang dipakai untuk menyelesaikan masalah pada suatu desain kode supaya lebih rapi dan juga mudah dikembangkan

1. FACTORY

```
public Ingredient createIngredientByType(GamePanel gp, IngredientType type) {
    final IngredientState defaultState = IngredientState.RAW;

    switch (type) {
        case BUN:
            return new Bun(gp, defaultState);
        case CHEESE:
            return new Cheese(gp, defaultState);
        case LETTUCE:
            return new Lettuce(gp, defaultState);
        case MEAT:
            return new Meat(gp, defaultState);
        case TOMATO:
            return new Tomato(gp, defaultState);
        default:
            return null;
    }
}
```

Factory Design Pattern memusatkan pembuatan objek agar kode lebih rapi dan mudah dikembangkan. method `createIngredientByType` membuat berbagai ingredient berdasarkan `IngredientType` tanpa mengekspos detail pembuatannya.

2. SINGLETON

```
public class OrderManager implements GameSubject {
    private static OrderManager instance;

    // Private constructor prevents external instantiation
    private OrderManager() {
        initBurgerRecipes();
    }

    // Global access point
    public static OrderManager getInstance() {
        if (instance == null) {
            instance = new OrderManager();
        }
        return instance;
    }

    // Business methods
    public void generateOrder() { ... }
    public boolean validateDish(List<Preparable> plateContents) {
```

Singleton Design Pattern memastikan hanya ada satu instance dari suatu class yang digunakan secara global di seluruh aplikasi. `OrderManager` dibuat dengan constructor private dan method `getInstance()` sebagai satu-satunya cara mengakses instance tersebut, sehingga seluruh pengelolaan order terpusat dan konsisten tanpa risiko duplikasi objek.



3. COMMAND

```
public interface Command {  
    void execute(Chef chef);  
}
```

```
public class InteractCommand implements Command {  
    private GamePanel gamePanel;  
    private Chef chef;  
  
    public InteractCommand(GamePanel gp, Chef chef) {  
        this.gamePanel = gp;  
        this.chef = chef;  
    }  
  
    @Override  
    public void execute(Chef chef) {  
        Direction direction = chef.getDirection();  
        Station station = gamePanel.collisionChecker.checkStation(  
            direction, chef);  
  
        if (station != null) {  
            station.interact(chef);  
        }  
    }  
}
```

Command Design Pattern memisahkan aksi sebagai objek perintah supaya input dan logika eksekusinya tidak saling bergantung. Pada gambar, interface `Command` mendefinisikan method `execute`, lalu `InteractCommand` mengimplementasikannya untuk menangani aksi chef saat berinteraksi dengan station, sehingga tiap aksi bisa dikelola, diganti, atau dikembangkan tanpa mengubah alur utama game.

4. STRATEGY

```
// Choose strategy at runtime  
Preparable lettuce = new Lettuce(gp, IngredientState.RAW);  
  
// Use cutting strategy for RAW ingredients  
ProcessingStrategy strategy = new CuttingStrategy(lettuce, 3000);  
strategy.process(100); // Process 100ms  
float progress = ((CuttingStrategy) strategy).getProgress(); // 3.3%  
  
// Use cooking strategy for cooking  
Preparable meat = new Meat(gp, IngredientState.CHOPPED);  
ProcessingStrategy cookStrategy = new CookingStrategy(meat, 12000, 24000);  
cookStrategy.process(1000); // Process 1 second  
String type = cookStrategy.getProcessType(); // "COOKING"  
  
// Strategies are interchangeable  
ProcessingStrategy currentStrategy;  
if (needsCutting) {  
    currentStrategy = new CuttingStrategy(item, 3000);  
} else {  
    currentStrategy = new CookingStrategy(item, 12000, 24000);  
}  
currentStrategy.process(deltaTime);
```

Strategy Design Pattern memungkinkan perilaku atau algoritma diganti secara dinamis tanpa mengubah struktur utama program. Pada gambar, proses ingredient seperti memotong atau memasak ditentukan oleh strategy yang berbeda (`CuttingStrategy` dan `CookingStrategy`), sehingga setiap bahan bisa diproses dengan cara yang sesuai tanpa logika bercabang yang rumit.

5. OBSERVER

```
// Update order timers  
for (Order o : activeOrders) {  
    o.updateTimer(deltaTime);  
    if (o.isExpired()) {  
        activeOrders.remove(o);  
        expiredOrders++;  
        System.out.println("[ORDER] ORDER EXPIRED: " + o.getRecipe().getName() + " (-10 points)");  
        score -= 10; // Allow negative score for lose condition  
        System.out.println("[ORDER] New score: " + score);  
  
        // OBSERVER PATTERN: Notify observers of order expiration  
        for (GameObserver observer : observers) {  
            observer.onOrderExpired(-10);  
        }  
        notifyObservers();  
    }  
}
```

Observer Design Pattern memungkinkan satu perubahan state langsung memberi tahu banyak bagian lain yang bergantung padanya. Pada gambar, saat order kedaluwarsa, sistem mengurangi skor lalu memberi notifikasi ke seluruh observer melalui `onOrderExpired`, sehingga efek perubahan bisa tersebar otomatis tanpa saling bergantung langsung.

PEMBAGIAN TUGAS



RAFEYFA DEA

CODING

1. Membuat Chef
2. Membuat GameMap
3. Membuat tampilan GUI untuk items, menu, dan chef

DOKUMEN

1. Membuat Booklet
2. Membuat Class Diagram



ZAHRA ZAHIDAH

CODING

1. Membuat package Main
2. Membuat tampilan GUI untuk items

DOKUMEN

1. Membuat Booklet
2. Membuat Class Diagram
3. Finalisasi Class Diagram



ZAHRAH NUR A

CODING

1. Membuat package Stations
2. Membuat ingredients, kitchen utensil, dan dish
3. Membuat tampilan GUI untuk map dan menu

DOKUMEN

1. Membuat Booklet
2. Membuat Class Diagram

ATHAR RAMADHANA

CODING

1. Membuat semua Actions
2. Membuat semua Orders
3. Membuat semua Recipe
4. Menggabungkan semua source code

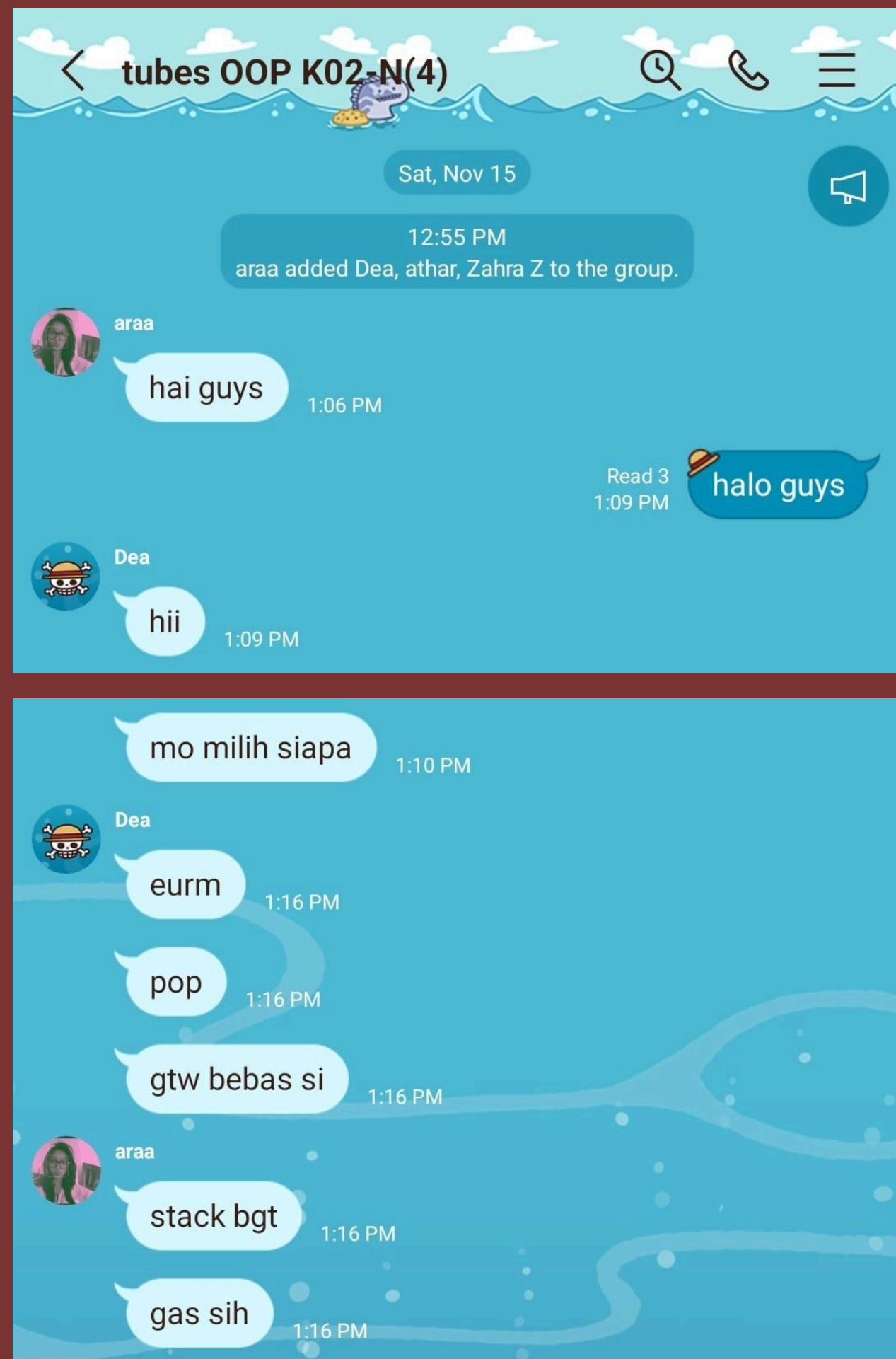
DOKUMEN

1. Membuat Class Diagram



BEHIND THE SCENES

15 November 2025



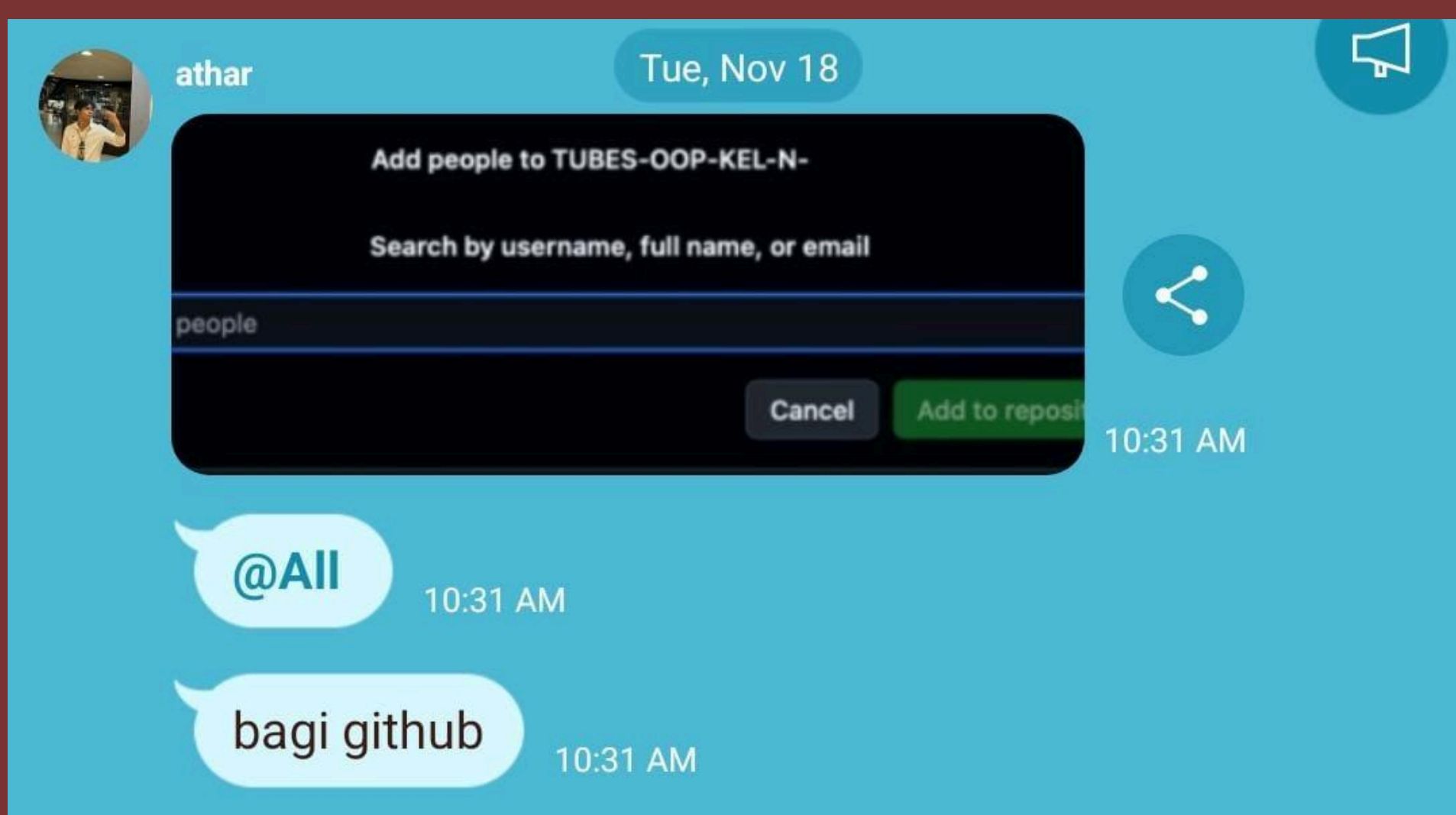
Berawal dari Ara ngechat dan nyulik kita semua ke grup. Pada hari itu juga kami memilih asisten secara random.



Ternyata dapat Kak Jendra! :D

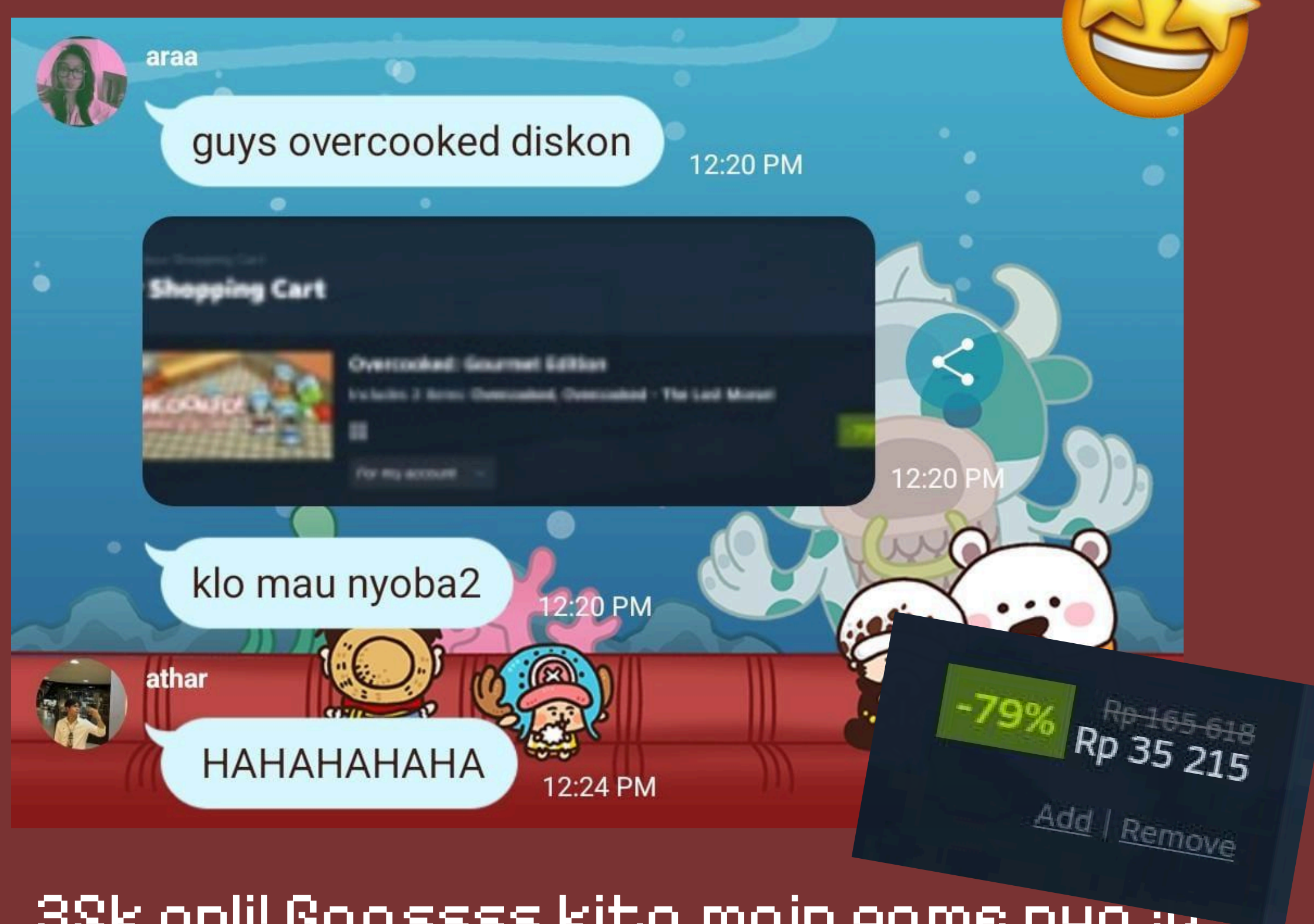
18 November 2025

Lanjut kita bikin repo dan add github masing masing {dikelola oleh Athar}
{makasih Athar}

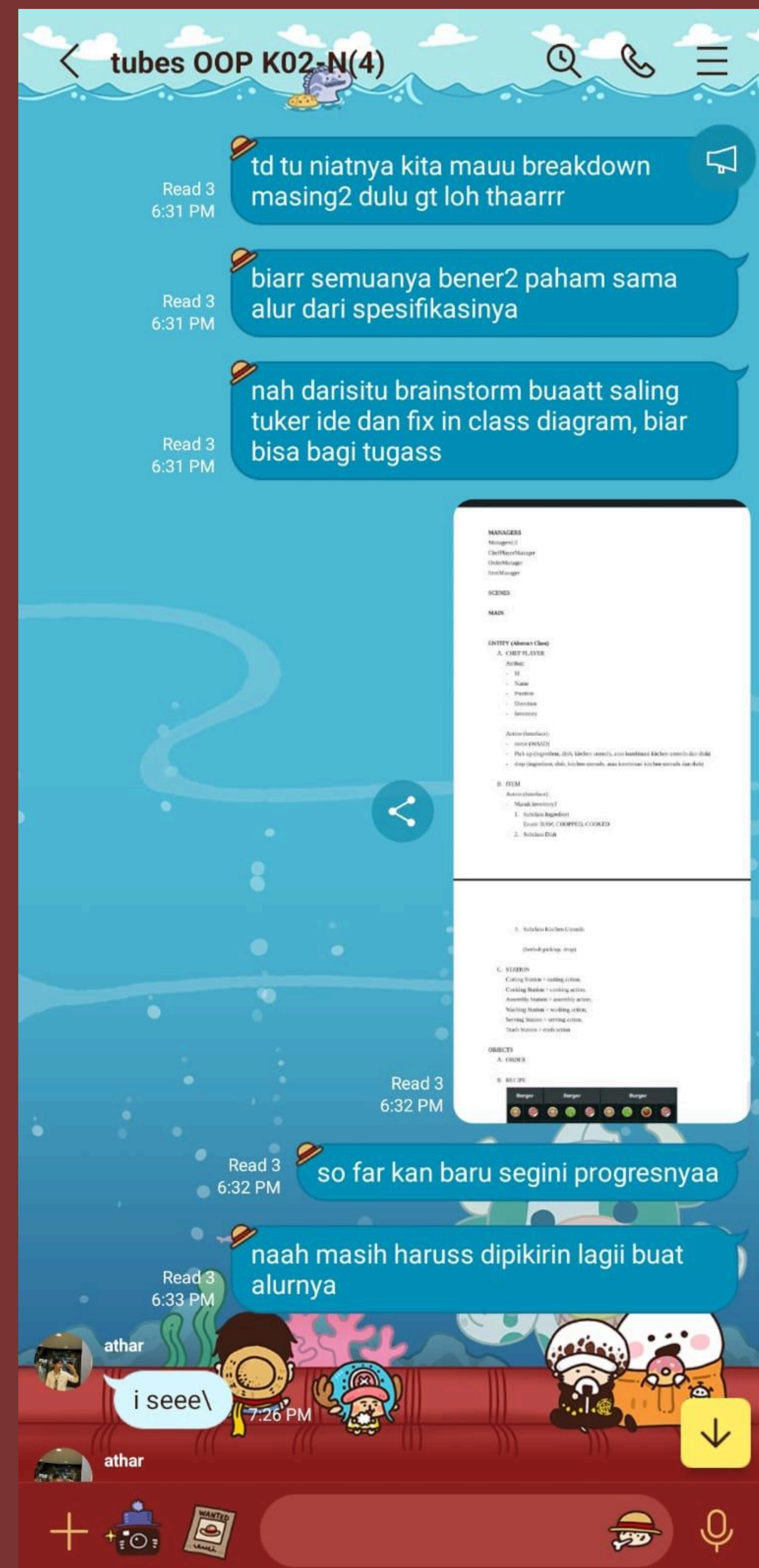


Masih di hari yang sama, Ara Dea Jay memutuskan untuk ketemu dan ngobrolin Milestone 1 buat bikin class diagram. Dari situ ada progres buat gambaran game dan alurnya.

Eh tapi tiba-tiba...

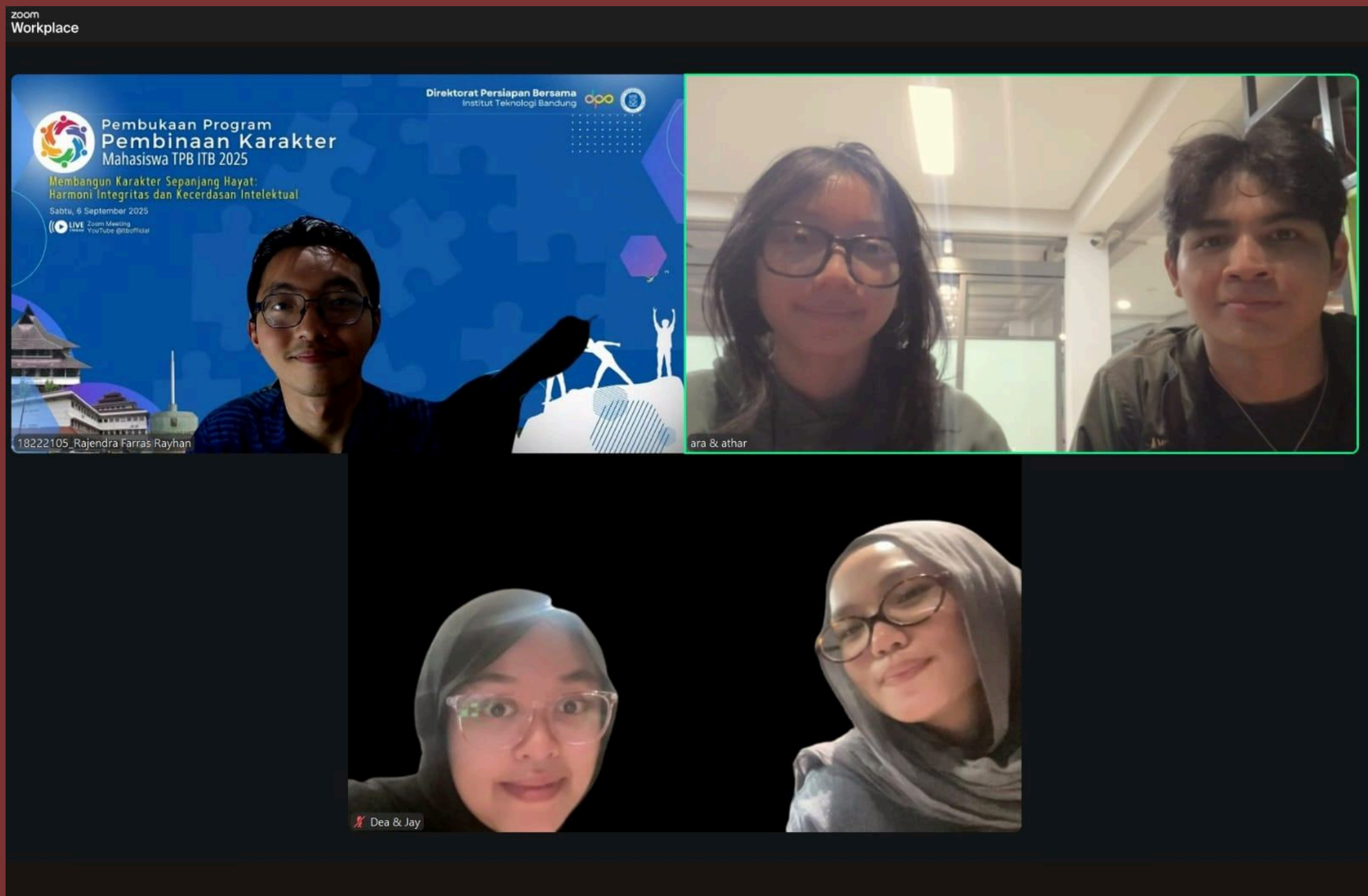


38k onli! Gaassss kita main game nya :v

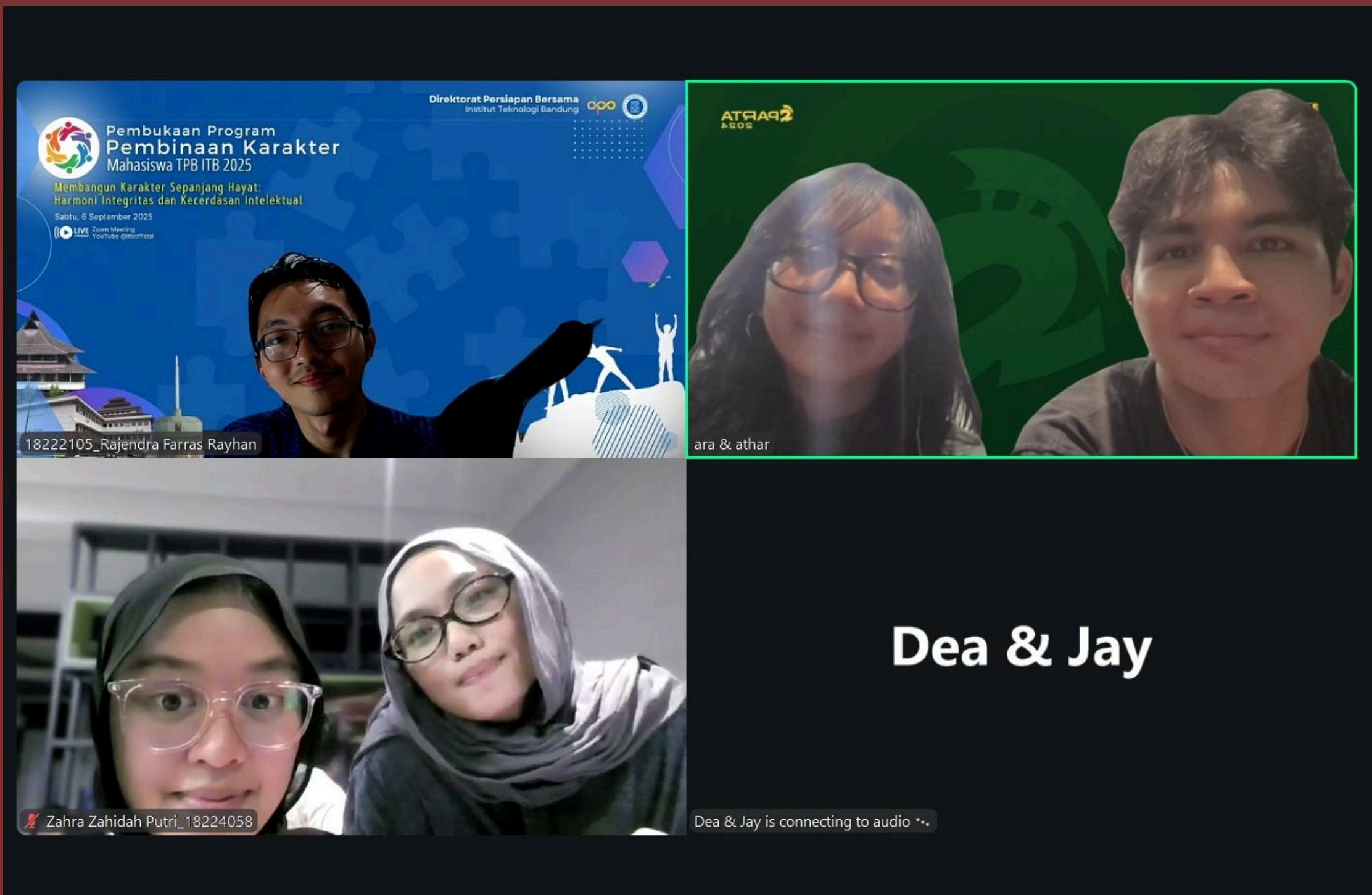


20 November 2025

Asistensi pertama buat Milestone satuuu!

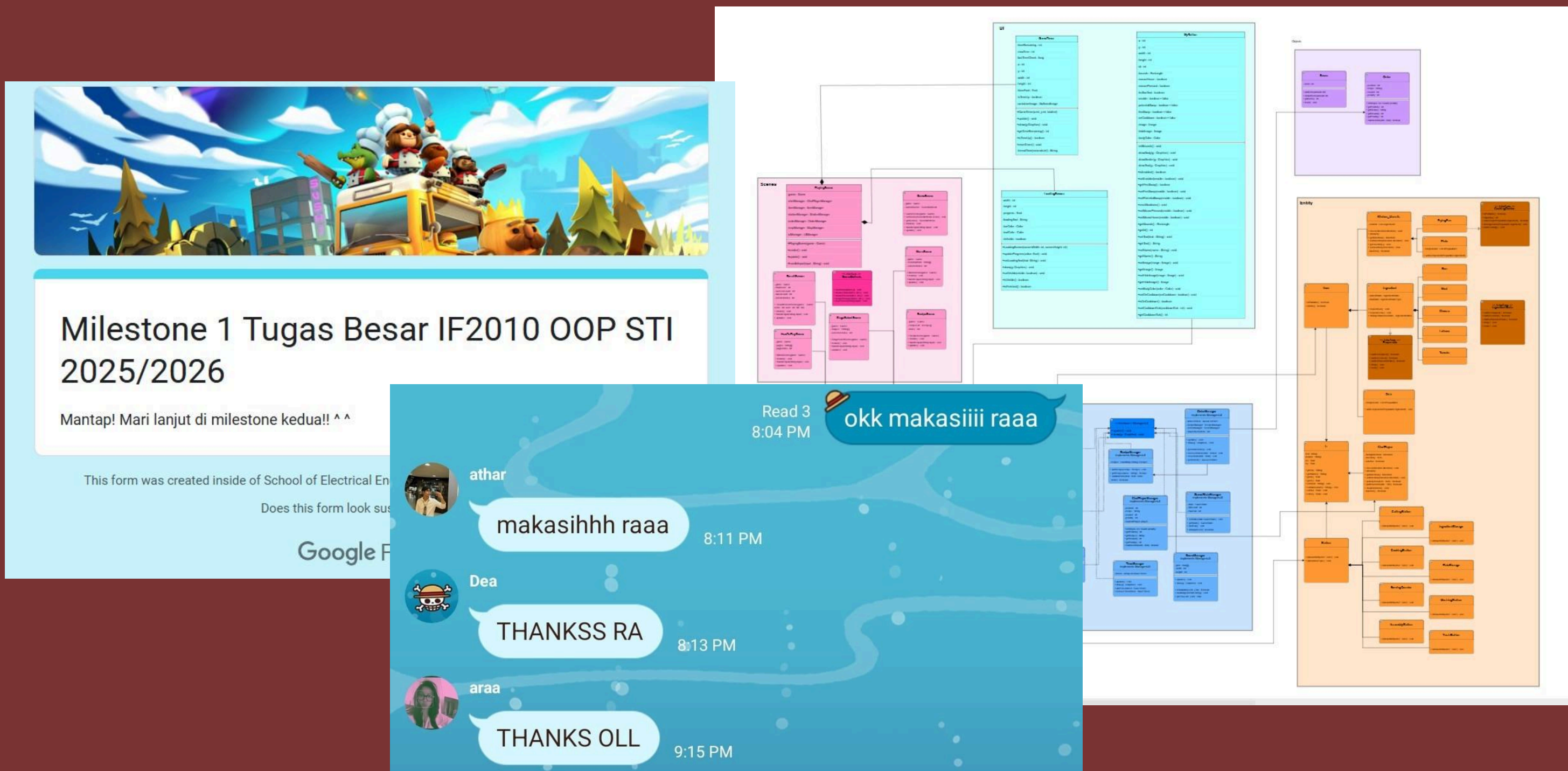


Dokum awal

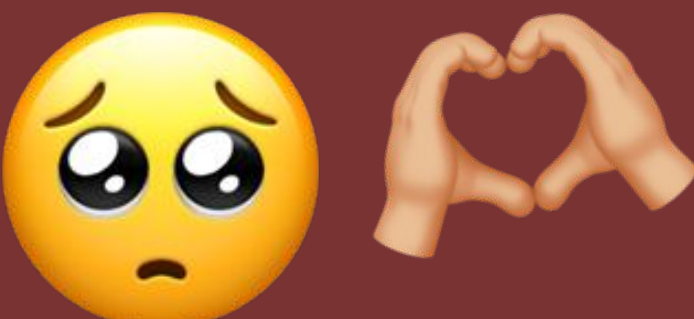


Dokum akhir

21 November 2025



Alhamdulillah
beres nih
Milestone satu



24 November 2025

Pembagian Tugas OOP

お使いのブラウザで JavaScript が有効になっていないため、このファイルは開けま...

ini td gw baru bikin kosongannya

No	Pengerjaan	PIC	Soft Deadline	Status
1	[1] HUD_OrderCard_Background (Panel to hold the recipe)	Athar	08/12/2025	☐
2	[1] HUD_OrderCard_TimerBar (Green/Yellow/Red progress bar)	Athar	08/12/2025	☐
3	[1] HUD_Score_Counter (Icon + Number font)	Athar	08/12/2025	☐
4	[1] HUD_Level_Timer (Main clock at top of screen)	Athar	08/12/2025	☐
5	[1] HUD_Star_Meter_Empty (Background bar)	Athar	08/12/2025	☐
6	[1] HUD_Star_Meter_Fill (Fill bar)	Athar	08/12/2025	☐
7	[1] HUD_Star_Icon_Active (Gold star)	Athar	08/12/2025	☒
8	[1] HUD_Star_Icon_Inactive (Gray/Empty star)	Athar	08/12/2025	☒
9	[1] HUD_Coin_Icon	Athar	08/12/2025	☒
10	[1] Menu_Pause_Background	Athar	08/12/2025	☐
11	[1] Menu_Button_Normal	Athar	08/12/2025	☐
12	[1] Menu_Button_Hover	Athar	08/12/2025	☒
13	[1] Screen_LevelComplete_Success (Background)	Athar	08/12/2025	☐
14	[1] Screen_LevelComplete_Fail (Background)	Athar	08/12/2025	☐
15	[1] Ind_PlayerMarker_P1 (Arrow/icon above Kirby P1)	Athar	08/12/2025	☒
16	[1] Ind_PlayerMarker_P2 (Arrow/icon above Kirby P2)	Athar	08/12/2025	☒
17	[1] Ind_Selection_Highlight (Square outline for current life target)	Athar	08/12/2025	☒
18	[1] Ind_Progress_Circle_Back (Empty circle background)	Athar	08/12/2025	☐

Pembagian Tugas OOP

No	Pengerjaan	PIC	Soft Deadline	Status
1	Recipe and Order	Athar	28/11/2025	☐
2	GUI	Athar	28/11/2025	☐
3	Items	Ara	28/11/2025	☒
4	Stations	Ara	28/11/2025	☐
5	Map	Dea	28/11/2025	☐
6	Chef	Dea	28/11/2025	☐
7	Manager	Jay	28/11/2025	☐
8	Main	Jay	28/11/2025	☐

TUBES-OOP-KEL-N

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

main 18 Branches 0 Tags

Go to file

Code

About

No description, website, or topics provided.

Readme

Activity

1 star

0 watching

0 forks

Report repository

Releases

No releases published

Create a new release

Packages

28 November 2025

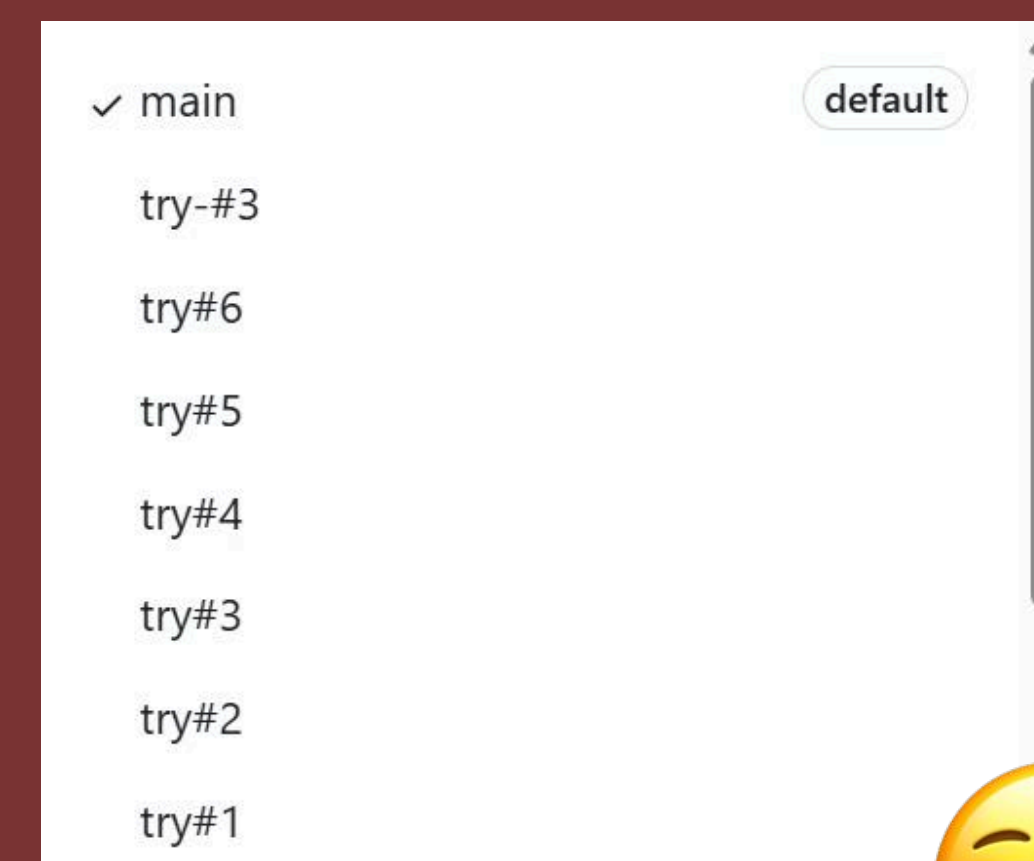
Athar lagi di
Jogja guys
jadi gaikut
kerkom



5 Desember 2025

Oke udah mulai
banyak progres
masing-masing nih,
saatnya kita
merge dan
lahirnya branch
try#1

Spoiler:



Ended up ada sampe try#6



9 Desember 2025

Asistensi kedua buat Milestone dua. zzz



Dokum awal



Ini kita masih di cc barat
(baru beres asistensi)



Dokum akhir



Pindah ke
SUGU deh!!



Ara nguap
karena cape

10 Desember 2025

Di Jabaranan ari jam 11 pagi
sampai 10 malem oke juga



11 Desember 2025



Dengan muka udah capek, masih sempet-
sempetnya photobox dulu LOLZIES

12 Desember 2025

LAST DANCEEEEE!!!! 🕺🕺

© duwatiga paskal

YUK BERESIN BERESIN
biar cepet kelar



A pixel art illustration of Kirby in a kitchen. Kirby, a pink, round character with a large head, is wearing a white chef's hat and a white apron. He is standing behind a wooden counter, looking surprised or excited with wide eyes and an open mouth. On the counter, there are several items: a large pot with a bright orange and yellow flame underneath it, a plate with a green vegetable and a brown sauce, and a small bowl. To the left of Kirby, there is a wooden rack with a spatula and a whisk hanging from it. Above the rack, there is a small jar with a brown substance inside. To the right of Kirby, there is a wooden stove with a fire burning in the pot. On the floor, there is a large pot with a bright orange and yellow flame underneath it, a plate with a green vegetable and a brown sauce, and a small bowl. The background is a simple wooden wall with a wooden cabinet on the right side.

THANK YOU!

"Poyo... even Kirby knows that true power isn't what you inhale,
but what you choose to care for."